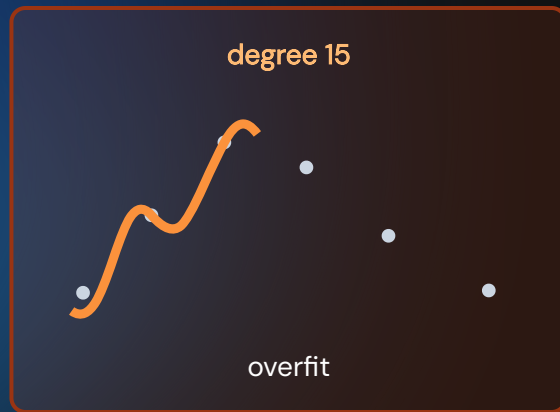
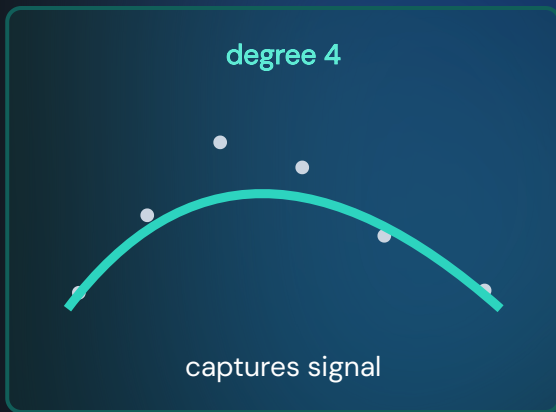
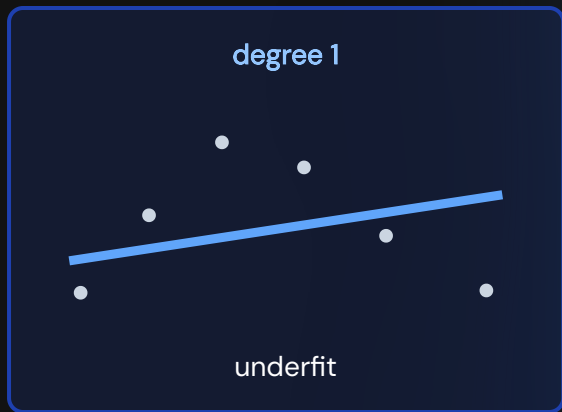


Polynomial Regression

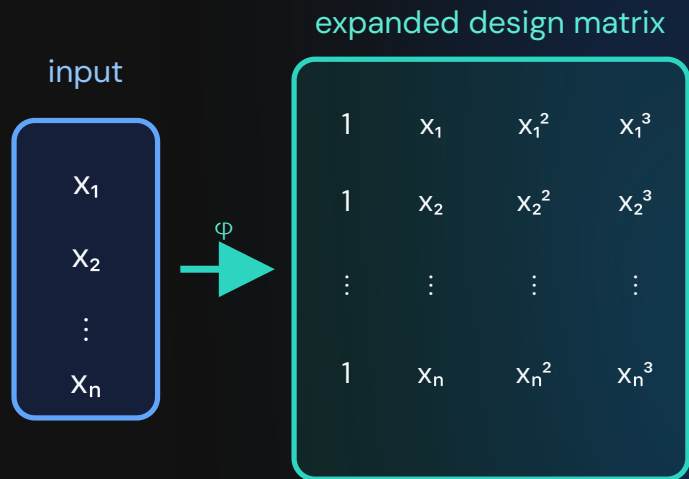
Ridge, Lasso, and the Price of Flexibility

Fit curves with a model that is still linear in its parameters



Topic 2 of Supervised Learning: Regression

Expand the Features, Keep the Linear Model



$$\hat{y} = w_0 + w_1x + w_2x^2 + \cdots + w_px^p = \phi(x)^\top w$$

- nonlinear in the original input x
- linear in the learned parameters w
- same least-squares objective
- same closed-form or gradient-based solvers

“Linear model” describes how parameters enter the equation—not whether the plotted prediction is a straight line.

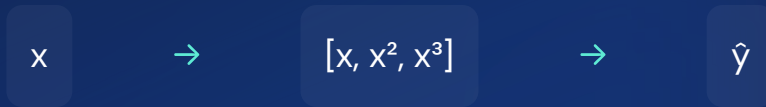
Polynomial Features in scikit-learn

```
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures

rng = np.random.default_rng(0)
X = rng.uniform(-3, 3, 30).reshape(-1, 1)
y = .5 * X[:, 0]**3 - X[:, 0]**2 + rng.normal(0, 2, 30)

model = make_pipeline(
    PolynomialFeatures(degree=3, include_bias=False),
    LinearRegression()
).fit(X, y)
```

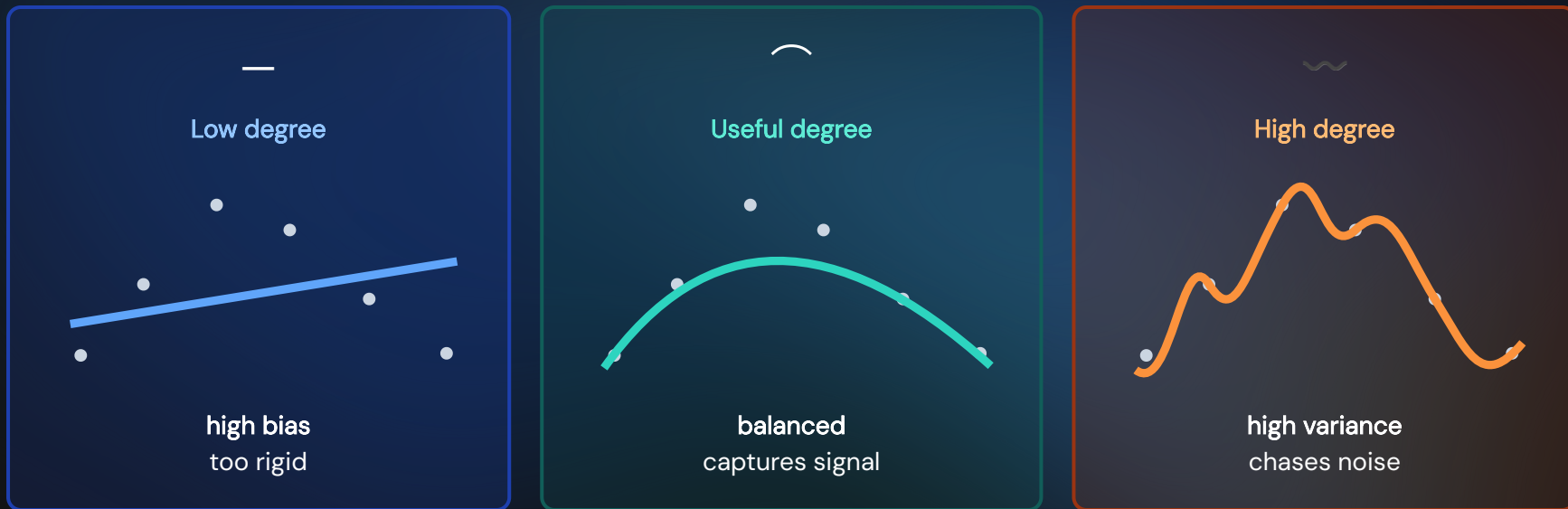
Pipeline data flow



- transformation is learned only where needed
- prediction automatically repeats every step
- `include_bias=False` avoids duplicating the intercept

Keep preprocessing inside the pipeline so cross-validation cannot leak information.

Degree Is a Complexity Dial

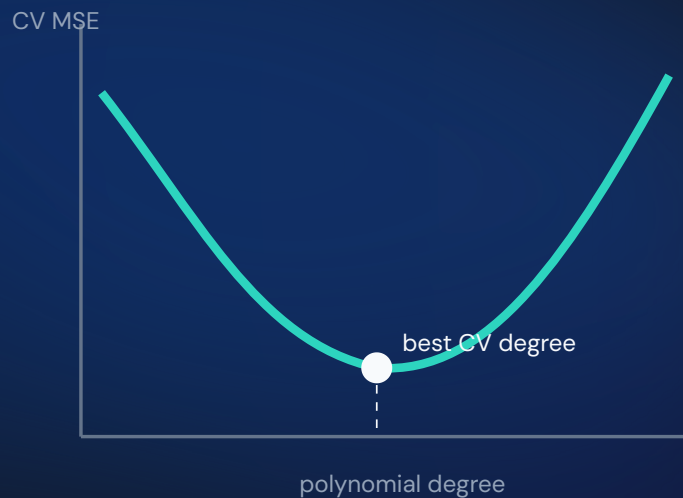


Training error keeps falling; validation error usually forms a **U-shape**.

Let Cross-Validation Choose the Degree

```
from sklearn.model_selection import cross_val_score

degrees = [1, 2, 3, 4, 9, 15]
for degree in degrees:
    model = make_pipeline(
        PolynomialFeatures(degree, include_bias=False),
        LinearRegression()
    )
    mse = -cross_val_score(
        model, X, y, cv=5, scoring="neg_mean_squared_error"
    ).mean()
    print(degree, round(mse, 2))
```



Choose from held-out folds—not the prettiest training curve.

Ridge: Add One Term, Gain Stability

$$\begin{aligned}J_{\text{ridge}}(w) &= \|y - Xw\|_2^2 + \lambda \|w\|_2^2 \\ \nabla_w J_{\text{ridge}}(w) &= -2X^\top y + 2X^\top Xw + 2\lambda w \\ 0 &= -2X^\top y + 2(X^\top X + \lambda I)\hat{w}\end{aligned}$$

$$\hat{w}_{\text{ridge}} = (X^\top X + \lambda I)^{-1} X^\top y$$

$\lambda = 0$

ordinary least squares

larger λ

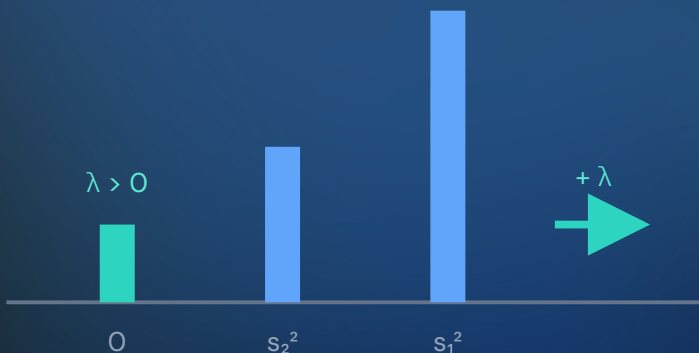
stronger shrinkage

intercept

normally left unpenalized

Derivation assumes centered data or a separate, unpenalized intercept.

Why Ridge Fixes Singularity



$$X^\top X = V \operatorname{diag}(s_1^2, \dots, s_d^2) V^\top$$

$$X^\top X + \lambda I = V \operatorname{diag}(s_1^2 + \lambda, \dots, s_d^2 + \lambda) V^\top$$

- every eigenvalue shifts upward by λ
- zero becomes strictly positive
- the inverse now exists for $\lambda > 0$
- correlated polynomial columns become manageable

Ridge Trades Variance for Bias

$$\hat{w}_{\text{ridge}} = V \operatorname{diag}\left(\frac{s_i^2}{s_i^2 + \lambda}\right) V^\top \hat{w}_{\text{OLS}}$$

- each OLS direction is shrunk by a factor

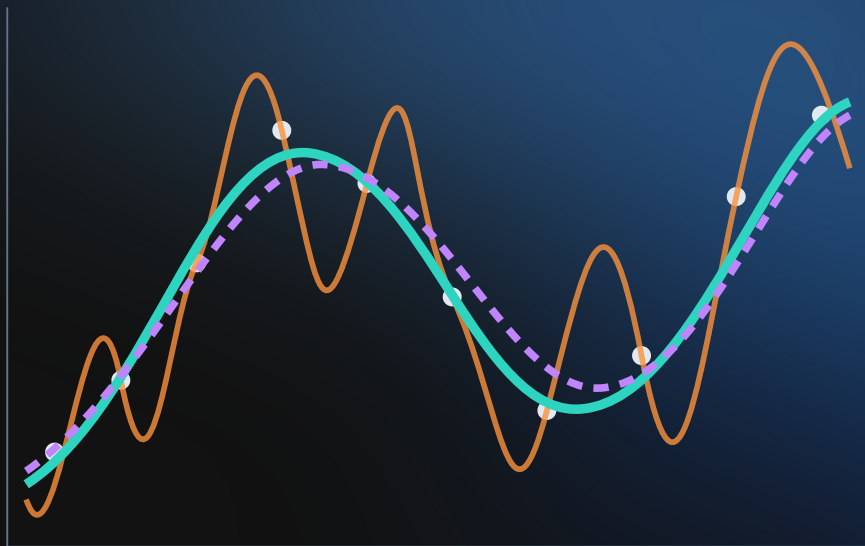
$$\frac{s_i^2}{s_i^2 + \lambda} \in (0, 1]$$

- small s_i (low-signal, high-variance direction) → shrunk hardest
- large s_i (high-signal direction) → barely shrunk
- shrinkage introduces bias, but removes far more variance than it adds

Why this is the bias–variance tradeoff

OLS is unbiased but can have huge variance along directions with small s_i , where a tiny amount of noise in y produces a huge change in $\hat{w}$. Ridge accepts a little bias along exactly those directions in exchange for a large reduction in variance — the same tradeoff degree controls for polynomial features, now controlled continuously by λ .

One Flexible Basis, Three Fits



Unregularized

wild coefficients, high variance

Ridge · L2

shrinks every term smoothly

Lasso · L1

can zero higher-order terms

Ridge Shrinks; Lasso Selects

Ridge · $\lambda \|w\|_2^2$

```
from sklearn.linear_model import Ridge

ridge = Ridge(alpha=5.0)
ridge.fit(X_poly, y)
print(abs(ridge.coef_).max())
```

Differentiable penalty; a stable closed-form solution exists.

Lasso · $\lambda \|w\|_1$

```
from sklearn.linear_model import Lasso

lasso = Lasso(alpha=.05, max_iter=20_000)
lasso.fit(X_poly, y)
print(np.count_nonzero(lasso.coef_))
```

Nondifferentiable at zero; solved iteratively, often with coordinate descent.

Ridge keeps all terms small; lasso can remove terms entirely.

Deriving Lasso's Sparsity: Soft-Thresholding

Assume an orthonormal design ($X^\top X = I$), so the objective separates coordinatewise:

$$J_{\text{lasso}}(w) = \text{const} + \sum_j \left[w_j^2 - 2w_j \hat{w}_j^{\text{OLS}} + \lambda |w_j| \right]$$

- minimize each term in w_j independently
- for $w_j \neq 0$, ordinary calculus applies: $2w_j - 2\hat{w}_j^{\text{OLS}} \pm \lambda = 0$
- for $w_j = 0$, the subgradient of $|w_j|$ spans $[-1, 1]$, so $w_j = 0$ is optimal whenever $|\hat{w}_j^{\text{OLS}}| \leq \lambda/2$

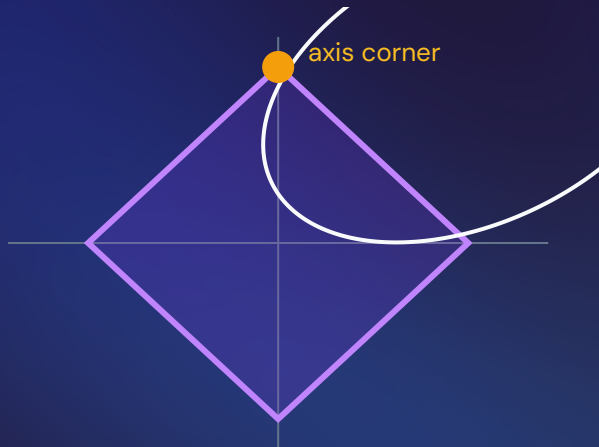
$$\hat{w}_j = \text{sign}(\hat{w}_j^{\text{OLS}}) \max\left(|\hat{w}_j^{\text{OLS}}| - \frac{\lambda}{2}, 0\right)$$

"Soft-thresholding": any OLS coefficient smaller than $\lambda/2$ in magnitude is pushed to exactly zero; larger ones are shrunk by a constant amount, not a constant factor.

Why L1 Finds Exact Zeros

Equivalent view: minimize the loss subject to a budget $\|w\|_1 \leq t$ (or $\|w\|_2^2 \leq t$) — the constraint form of the same penalized objective from the earlier slides.

L1 constraint



Corners lie on axes $\rightarrow$ one coordinate is exactly zero

L2 constraint



Smooth boundary $\rightarrow$ coefficients are usually small but nonzero

Scale Before You Regularize

$$x_j^{\text{scaled}} = \frac{x_j - \mu_j}{\sigma_j}$$

- penalties act on coefficient magnitude
- coefficient magnitude depends on feature units
- $x, x^2, \dots, x^p$ live on different scales
- standardization makes the penalty comparable

```
from sklearn.linear_model import RidgeCV
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures
from sklearn.preprocessing import StandardScaler
```

```
model = make_pipeline(
    PolynomialFeatures(9, include_bias=False),
    StandardScaler(),
    RidgeCV(alphas=np.logspace(-4, 4, 40), cv=5),
)
```

```
model.fit(X, y)
print(model[-1].alpha_)
```

The pipeline fits scaling separately inside every CV training fold.

The Regression Recipe



Features

raw or polynomial



Loss

squared error



Penalty

L2 or L1



Solver

direct or iterative

Flexible features create capacity; **cross-validation and regularization** control it.

Next: Supervised Learning — Classification